풀이 및 채점기준

전자전기컴퓨터공학부 1학년 2020년 1학기 40121-2 C프로그래밍 중간시험

(2020년 5월 8일 금요일 오전 10시~11시 50분; 총5쪽 5문제, 총점 100점)

(오픈북/온라인 시험)

백지에 "C프로그래밍 2020년 1학기 2반 중간시험"/학번/성명 기재 후, 풀이와 답을 기재 시험 종료 시, 답지를 촬영하거나 스캔하여 jpg 또는 pdf 파일로 만든 후, 하나의 zip 파일에 담아 파일명을 C2M(이름)(학번 마지막 자리 수).zip(예: C2M홍길동9.zip)으로 하고, 이를 반드시 학교 이메일 계정을 사용하여 아래 강사와 조교 이메일 주소로 두 사람 모두에게 송부

김용한 교수(yhkim@uos.ac.kr), 김정남 조교(wjdska0012@uos.ac.kr)

아래 모든 소스 프로그램은 인텔 CPU를 탑재한 컴퓨터의 윈도우10 OS 상에서 Visual Studio 2017 Debug 모드를 사용하여 빌드하고 실행하였다.

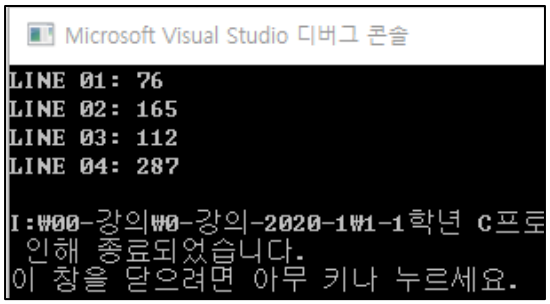
문제 1. (16점) 아래 소스 프로그램에 대한 실행 예(아래 우하단에 나와 있음)를 보고 소스 프로그램에서 가려진 부분의 내용(정수)과 출력 부분에서 가려진 내용(정수)을 문제 번호와 함께 써라.

```
#include <stdio.h>

int add1(int), add2(int), add3(int);
int num = [redacted];   <- 문제 1-1

void main(void)
{
    register int num = 10;
    int line = 1;

    printf("LINE %02d: %d\n", line++, add1(num) + add2(num));
    printf("LINE %02d: %d\n", line++, add3(add2(num)));
    {
        int num = 20;

        printf("LINE %02d: %d\n", line++, num = add2(num));
    }
    printf("LINE %02d: %d\n", line++, add3(num));
}
int add1(int num)
{
    num += 30;
    return num;
}
int add2(int val)
{
    static int num = [redacted];   <- 문제 1-2

    num += val;
    return num;
}
int add3(int val)
{
    num += add2(val);
    return num;
}
```

```
Microsoft Visual Studio 디버그 콘솔
LINE 01: 76
LINE 02: 165
LINE 03: 문제 1-3
LINE 04: 문제 1-4

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로
 인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 1 끝.

답:



문제 1-1의 답: 73

문제 1-2의 답: 26

문제 1-3의 답: 112

문제 1-4의 답: 287

채점 기준:

각 소문항 당 4점 배점. 부분 점수 없음

문제 1-1의 답과 문제 1-2의 답을 각기 x와 y라 하자. LINE 01의 경우, add1(num)의 반환값이 10+30=40이고, add2(num)의 반환값이 10+y이므로, 그 합은 y+50이 된다. 이를 출력값과 같다고 놓으면 y+50=76이 된다. 따라서 문제 1-2의 답은 y=76-50=26이다. 이 문장을 수행하고 나면, add2의 static num 값은 36이 된다. (아래 표 참고 요망)

LINE 02의 경우, add2(num)의 반환값은 10+36=46이고(이후 add2의 static num 값이 46임), add3 함수 내에서 add2(46)이 호출되므로 그 반환값은 46+46=92가 되고(이후 add2의 static num 값은 92임), global num 값은 x+92가 된다. 이 값을 출력값과 같다고 놓으면 x+92=165가 되어, 문제 1-1의 답은 x=165-92=73이 된다. (아래 표 참고 요망)

LINE 03의 경우, printf문 내의 add2(num)에서 num은 그 문장 바로 위에 선언된 num을 참조하므로, add2(num)은 실제 add2(20)이고 그 반환값은 92+20=112이며(이후 add2의 static num 값은 112임), 이 값은 해당 printf문이 속한 중괄호 내의 local 변수 num에 저장된다. 따라서 문제 1-3의 답은 112이다. 중괄호 내의 local 변수 num은 중괄호를 빠져나올 때 그 메모리가 해제된다.

LINE 04의 경우, printf문 내의 add3(num)에서 num은 main 함수의 local 변수인 num의 값이므로, add3(num)은 실제 add3(10)이다. add3 함수 내의 add2(val)의 반환값은 112+10=122가 되고(이후 add2의 static num 값은 122임), global num은 165+122=287이 되어 이 값이 출력된다 (아래 표 참고 요망) 따라서 문제 1-4의 답은 287이다.

	main의 local num	global num	add2의 static num
초기값	10	73	26
LINE 01 출력 후	10	73	36
LINE 02 출력 후	10	165	92
LINE 03 출력 후	10	165	112
LINE 04 출력 후	10	287	122

문제 2. (16점) 아래 소스 프로그램에 대한 실행 예(아래 우하단에 나와 있음)를 보고 소스 프로그램에서 가려진 부분의 내용(두 자리의 수임)을 문제 번호와 함께 써라.

```
#include <stdio.h>

int funcA(int num)
{
    return ~num + 1;
}

int funcB(int num)
{
    return ~(num - 1);
}

int funcC(int num)
{
    int mask = 1;

    do  {
        if (num & mask)
            break;
        mask <<= 1;
    } while (mask);
    for (mask <<= 1; mask; mask <<= 1)
        num ^= mask;

    return num;
}

void showResult(int num)
{
    static int i = 0;

    printf("LINE %02d: %4d\n", ++i, num);
}

void main(void)
{
    showResult(funcA(0x[redacted]));
    showResult(funcA(funcA(0x[redacted])));
    showResult(funcB(funcB(~0x[redacted] & 0xFF)));
    showResult(funcC(0x[redacted]));
}
```

문제 2-1 문제 2-2 문제 2-3 문제 2-4

Microsoft Visual Studio 디버그 콘솔

```
LINE 01: -170
LINE 02:  252
LINE 03:  252
LINE 04: -238

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로
 인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 2 끝.

<u>답</u>:

문제 2-1의 답: AA

문제 2-2의 답: FC

문제 2-3의 답: 03

문제 2-4의 답: EE

<u>채점 기준</u>:

각 소문항 당 4점 배점. 문제 2-3의 경우, 3이라고만 해도 맞는 답임. 다른 부분 점수 없음

<u>풀이</u>:

어떤 양의 정수 x를 -x로 바꾸려면 2의 보수를 취하면 된다. 만약 어떤 음의 정수 -x(x는 양수)를 양의 정수 x로 바꾸려면 역시 2의 보수를 취하면 된다. 즉 -(-x)=x가 된다. 어떤 수 x의 2의 보수 y란 것은 이진수로 x와 y를 표현한 후 이진법에 의해 x+y를 하였을 때 0(모든 비트가 0가 되며, MSB(Most Significant Bit, 최상위 비트)에서 발생하는 올림수 1은 나타낼 방법이 없으므로 삭제됨)이 되는 수이므로, x가 양수이든 음수이든 상관없이 그 2의 보수를 취하면 x의 부호를 바꿀 수 있다.

2의 보수를 취하는 방법으로서 아래 세 가지 방법은 동등하다.

(1) 1의 보수를 취한 후(모든 비트들을 반전시킨 후), 그 결과에 1을 더한다.
(2) 1을 뺀 후, 그 결과에 1의 보수를 취한다(모든 비트들을 반전시킨다).
(3) LSB(Least Significant Bit, 최하위 비트)부터 MSB 측으로 한 비트씩 읽어 내어, 처음으로 만나는 1까지의 비트들은 그 값을 그냥 두고, 그 다음 비트부터 MSB까지의 비트들을 반전시킨다.

(2)의 방법이 (1)의 방법과 동등함을 세 가지 경우로 나누어 보이고자 한다. 즉 (2)의 방법에 의해서도 (1)의 방법과 마찬가지로 정수의 부호를 바꿀 수 있음을 보이고자 한다. 이를 위해 주어진 수가 양수인 경우, 음수인 경우, 0인 경우의 세 가지 경우로 나누어 생각해 보자.

(i) 어떤 양의 정수 x가 있다고 할 때, 1을 뺀 후, 2의 보수를 취하면 -(x-1)이 될 것이다. 즉 -x+1이 된다는 것이므로, (2)의 방법에 의해, x로부터 1을 뺀 후 2의 보수를 취하는 대신 1의 보수만 취하면 즉, 마지막에 더해 주는 1을 더하지 않으면(이는 1을 뺀 후 2의 보수를 취해 얻은 결과에서 1을 빼는 것과 같으므로), (-x+1)-1=-x가 된다. 즉 x의 부호가 바뀐다.

(ii) 또 어떤 음의 정수 -y(y는 양수)가 있다고 할 때, 1을 뺀 후, 2의 보수를 취하면 -(-y-1)가 될 것이다. 즉 y+1이 된다는 것이므로, 2의 보수를 취하는 대신 1의 보수만 취하면(즉 2의 보수를 취하는 과정에서 마지막에 1을 더하는 과정을 생략하면) (y+1)-1=y가 된다. 즉 -y가 y로 되어 부호가 바뀐다.

(iii) 만약 주어진 수가 0이라면, 1을 뺀 후, 2의 보수를 취하면, -(-1)이 될 것이다. 즉 1이 된다는 것이므로, 주어진 수 0으로부터 1을 뺀 후 2의 보수를 취하는 대신 1의 보수만 취하면(즉 2의 보수를 취하는 과정에서 마지막에 1을 더하는 과정을 생략하면) 1-1=0이 된다. 즉 0은 0으로 되므로 부호가 바뀌는 것과 상관이 없다.

따라서 (2)의 방법은 (1)의 방법과 동등하다.

또 (3)의 방법이 (1)의 방법과 동등한 이유는 다음과 같다. 2의 보수 형태로 표현된 정수가 주어져 있다고 하고, LSB부터 읽어서 n번째 비트에서 처음으로 1이 등장한다고 하자. (1)의 방법에 따라 이 정수에 대한 2의 보수를 구하기 위해, 이 수에 대해 1의 보수를 취하면 LSB부터 n-1번째 비트까지의 비트들은 모두 1, 또 n번째 비트는 0으로 바뀌고 그 결과에 1을 더하면 LSB부터 n-1번째 비트까지는 모두 0으로 바뀌고 n번째 비트(현재 값이 0)에 올림수 1을 더해야 한다. 따라서 주어진 정수의 2의 보수는 LSB부터 n-1번째 비트까지는 모두 0이고 n번재 비트는 1이며, n+1번째 비트부터 MSB까지의 비트는 원래 비트를 반전시킨 값을 갖게 된다. 따라서 어떤 정수의 2의 보수를 취하는 한 가지 방법은 LSB부터 MSB 측으로 한 비트씩 읽어 내어, 처음으로 만나는 1까지의 비트들은 그 값을 그냥 두고, 그 다음 비트부터 MSB까지의 비트들을 반전시키는 것이다.

따라서 문제의 **funcA**, **funcB**, 그리고 **funcC**가 하는 일은 주어진 int 정수에 대해 2의 보수를 취하는 것 즉, 부호를 바꾸는 것으로서 동등하다.

main 함수의 첫 문장의 출력값은 -170이므로 funcA의 인자는 십진수로 -(-170)=170이며 이는 16진수로 AA이다. 문제2-1의 답은 AA이다.

main 함수의 두 번째 문장의 출력값은 252이므로 funcA의 인자는 십진수로 -(-(252))=252이며 이는 16진수로 FC이다. 문제 2-1의 답은 FC이다.

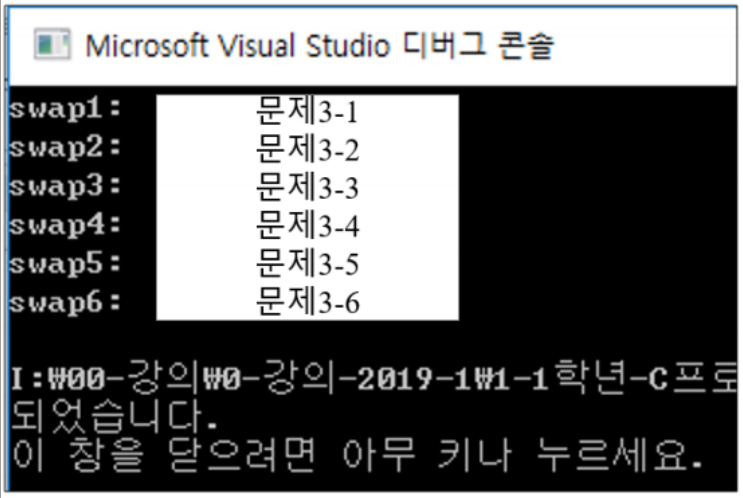
main 함수의 세 번째 문장의 출력값도 252이므로 문제 2-3의 답은 문제 2-2의 답 FC의 모든 비트를 반전시킨 값인 03이다. 문제 2-3의 답은 03이다.

main 함수의 네 번째 문장의 출력값은 -238이므로 funcC의 인자는 십진수로 -(-238)=238이며 이는 16진수로 EE이다. 문제 2-4의 답은 EE이다.

문제 3. (24점) 아래 소스 프로그램에 대한 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

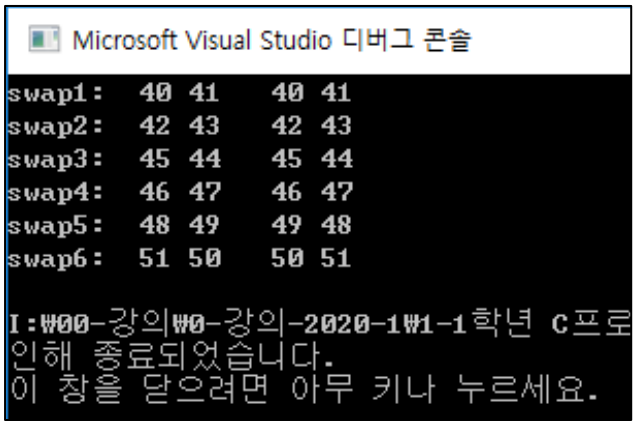
```c
#include <stdio.h>

void setNum(int a[])
{
    static int num = 40;

    a[0] = num++; a[1] = num++;
}
void swap1(int n1, int n2)
{
    int temp = n1;
    n1 = n2;
    n2 = temp;
}
void swap2(int *n1, int *n2)
{
    int *temp = n1;
    n1 = n2;
    n2 = temp;
}
void swap3(int *n1, int *n2)
{
    int temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}
void swap4(int **n1, int **n2)
{
    int **temp = n1;
    n1 = n2;
    n2 = temp;
}
void swap5(int **n1, int **n2)
{
    int *temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}
void swap6(int **n1, int **n2)
{
    int temp = **n1;
    **n1 = **n2;
    **n2 = temp;
}
void main(void)
{
    int num[] = { 0, 0 }, *p[] = { &num[0], &num[1] };

    setNum(num); swap1(num[0], num[1]);
    printf("swap1:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);

    setNum(num); swap2(p[0], p[1]);
    printf("swap2:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap3(p[0], p[1]);
    printf("swap3:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);

    setNum(num); swap4(p, p + 1);
    printf("swap4:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap5(p, p + 1);
    printf("swap5:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap6(p, p + 1);
    printf("swap6:  %d %d   %d %d\n", num[0], num[1], *p[0], *p[1]);
}
```

아래는 문제 3의 실행 예임



문제 3 끝.

답:



문제 3-1의 답: 40 41 40 41

문제 3-2의 답: 42 43 42 43

문제 3-3의 답: 45 44 45 44

문제 3-4의 답: 46 47 46 47

문제 3-5의 답: 48 49 49 48

문제 3-6의 답: 51 50 50 51

채점 기준:

각 값 당 1점(각 소문항 당 4점) 배점. 부분 점수 없음

풀이:

setNum 함수 내의 static 변수 num의 값은 처음 setNum 함수가 호출되었을 때 40으로 한 번 초기화되며, 이 후 setNum 함수가 다시 호출되었을 때 또 초기화되지는 않는다. 이는 일반적인 static 변수에 대한 초기화 과정이다. setNum 함수가 호출될 때마다, main 함수의 num 배열의 각 요소에는 setNum 함수 내의 static 변수 num 값에서부터 각기 하나 및 둘 증가된 값이 저장된다.

main 함수 내의 포인터 배열 p에는 main 함수의 num 배열의 첫 요소와 둘째 요소의 주소값이 저장된다.

swap1 함수의 경우, call-by-value에 의한 함수이므로 이를 호출한 함수(여기서는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-1의 답은 40 41 40 41이다.

swap2 함수의 경우 call-by-reference에 의한 함수이긴 하지만, 내부적으로 매개변수의 값(이를 호출한 함수로부터 copy하여 넘겨받은 주소 값)만 서로 바꿀 뿐이므로 이를 호출한 함수(여기서는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-2의 답은 42 43 42 43이다.

swap3 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배열 num의 두 요소의 주소지)에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 num의 두 요소의 값)을 서로 바꾼다. 따라서 문제 3-3의 전반부의 답은 44 45이다. 문제 3-3의 후반부의 답의 경우, 배열 p에 저장된 주소값이 바뀐 것은 아니므로, p의 각 요소는 여전히 배열 num의 각 요소를 가리키고 있으므로, 전반부와 같은 값인 44 45가 답이 된다. 따라서 문제 3-3의 답은 45 44 45 44이다.

swap4 함수의 경우 call-by-reference에 의한 함수이긴 하지만, 내부적으로 매개변수의 값(이를 호출한 함수로부터 copy하여 넘겨받은 주소 값)만 서로 바꿀 뿐이므로 이를 호출한 함수(여기서

는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-4의 답은 46 47 46 47이다.

swap5 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배열 p의 두 요소의 주소지)에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 p의 두 요소의 값)을 서로 바꾼다. 따라서 배열 p의 첫 요소는 num[1]를 가리키고, 두 번째 요소는 num[0]을 가리키는 것으로 바뀌어 있다. 그러므로 문제 3-5의 전반부의 답은 num[0]와 num[1]를 직접적으로 읽은 48 49이고, 후반부의 답은 배열 p를 통해 num[1]와 num[0]를 순차적으로 읽은 것이므로 49 48이다. 따라서 문제 3-5의 답은 48 49 49 48이다.

swap6 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배열 p의 두 요소의 주소지)에 저장된 값 즉 main 함수의 배열 num의 두 요소의 주소지에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 num의 두 요소의 값)을 서로 바꾼다. 문제 3-6의 전반부의 답은 값이 바뀐 num[0]와 num[1]를 직접적으로 읽은 51 50이다. 후반부의 답은, 현재 배열 p의 첫 요소는 num[1]을, 두 번째 요소는 num[0]를 가리키고 있는 상태(swap5 함수 호출 이후, 이러한 상태가 되었음에 주의)이므로, 배열 p를 통해 num[1]와 num[0]를 순차적으로 읽은 것이므로 그 출력 결과는 50 51이다. 따라서 따라서 문제 3-6의 답은 51 50 50 51이다.

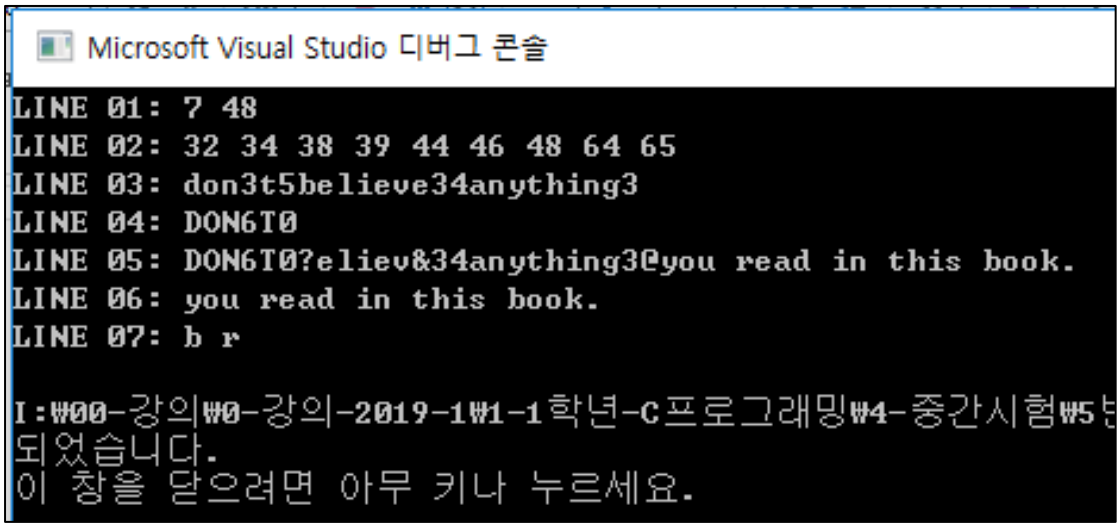
문제 4. (24점) 아래 소스 프로그램에 대한 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

```c
#include <stdio.h>

void main(void)
{
    char str[] = "Don\'t believe \"anything\" you read in this book.";
    char str2[80] = { 0 };
    char *ps, *pd;
    int i, count = 0, line = 1, len = 0;

    for (i = 0; str[i] != 0; i++)
        if (str[i] == ' ')
            count++;
    printf("LINE %02d: %d %d \n", line++, count, len = ++i);

    printf("LINE %02d: %d %d %d %d %d %d %d %d %d\n",
        line++, ' ', '\"', '&', '\'', ',', '.', '0', '@', 'A');

    for (str[len /= 2] = i = 0; str[i]; i++)
        if (str[i] < '0')
            str[i] = '0' + i % 10;
        else if (str[i] < 'a')
            str[i] += 'a' - 'A';
    printf("LINE %02d: %s\n", line++, str);

    for (str[len /= 4] = i = 0; str[i]; i++)
        if (str[i] <= '9')
            str[i] = '0'+(str[i] - '0') * 2 % 10;
        else if (str[i] > 'Z') // Zz
            str[i] -= 'a' - 'A';
    printf("LINE %02d: %s\n", line++, str);

    str[len] = '?'; str[len *= 2] = '&'; str[len *= 2] = 0x40;
    printf("LINE %02d: %s\n", line++, str);

    for (i = len * 2 - 1; i >= 0; i--)
        if (str[i]  == '@')
            break;
    for (ps = &str[i] + 1, pd = str2; *ps; )
        *pd++ = *ps++;
    printf("LINE %02d: %s\n", line++, str2);

    printf("LINE %02d: %c %c\n", line++, *(pd-- - 5),  str2[4]);
}
```

```
Microsoft Visual Studio 디버그 콘솔
LINE 01:                    문제 4-1
LINE 02: 32 34 38 39 44 46 48 64 65
LINE 03:                    문제 4-2
LINE 04:                    문제 4-3
LINE 05:                    문제 4-4
LINE 06:                    문제 4-5
LINE 07:                    문제 4-6

I:₩00-강의₩0-강의-2019-1₩1-1학년-C프로그래밍₩4-중간시험₩5번
되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 4 끝.

답:



문제 4-1의 답: `7 48`

문제 4-2의 답: `don3t5believe34anything3`

문제 4-3의 답: `DON6T0`

문제 4-4의 답: `DON6T0?eliev&34anything3@you read in this book.`

문제 4-5의 답: `you read in this book.`

문제 4-6의 답: `b r`

채점 기준:

각 소문항 당 4점 배점. 문제 4-1 및 문제 4-6의 경우, 각 값 당 2점 배점. 문제 4-2의 경우, 첫 글자 d에 2점, 나머지 부분에 2점 배점. 나머지 문제들의 경우, 전체 라인의 내용이 모두 맞아야 정답으로 인정됨. 다른 부분 점수 없음

풀이:

`"Don\'t believe \"anything\" you read in this book."`에 포함된 빈 칸은 모두 7개이고, 눈에 보이는 글자(빈 칸 포함)는 47글자이고, 문자열 마지막에 널 문자가 하나 더 있으므로 str 배열의 글자 수는 모두 48바이트이다. 눈에 보이는 글자를 셀 때, \'와 \"은 escape sequence로서 각기 작은 따옴표 및 큰 따옴표 한 글자인 것으로 세야 한다. 따라서 48바이트가 str 배열에 저장되어 있으므로, LINE 01에서는 7 48이 출력된다. 따라서 문제 4-1의 답은 `7 48`이다.

이후 `for (str[len /= 2] = i = 0; str[i]; i++)`에서 str 배열의 글자 수는 널 문자를 제외하고 24자인 것으로 바뀐다. 즉 str에 저장된 문자열의 24자 다음 부분이 잘린다. 이는 str[48/2=24]에 해당하는 요소 즉 25번째 글자 자리에 널 문자가 저장되기 때문이다. LINE 02의 출력 내용으로부터, `' '`, `'\"'`, `'&'`, `'\''`, `','`, `'.'`, `'0'`, `'@'`, `'A'`의 값은 각기 32, 34, 38, 39, 44, 46, 48, 64, 65이므로, `'0'` 보다 그 코드 값이 작은 글자들 즉, `' '`, `'\"'`, `'\''`은 `str[i] = '0' + i % 10;` 문장에 의해 모두 다른 글자로 바뀐다. 각 글자는 str 배열 내에서의 등장 위치 값을 10으로 나눈 나머지를 `'0'`에 더해 얻는 코드 값에 해당하는 글자로 바뀐다. 또 `'0'` 보다 그 코드 값이 작은 글자들을 제외하고 `'a'` 보다 그 코드 값이 작은 글자로서 str 배열에 저장된 잘라진 문자열에 등장하는 글자는 대문자뿐이므로 D는 d로 바뀐다. 따라서 문제 4-2의 답은 `don3t5believe34anything3`이다.

이후 `for (str[len /= 4] = i = 0; str[i]; i++)`에서 str 배열의 글자 수는 널 문자를 제외하고

6자인 것으로 바뀐다(len 값도 6으로 바뀐다). 그 결과 str 배열에 저장된 잘라진 문자열에 포함된 글자 중, '9' 보다 그 코드 값이 작거나 같은 글자는 아라비아 숫자에 해당하므로 이들이 다른 글자로 바뀌게 된다. 각 아라비아 숫자에 해당하는 글자는 그 수를 2배한 후 10으로 나눈 나머지에 해당하는 아라비아 숫자에 해당하는 글자로 바뀐다. '9' 보다 그 코드 값이 작거나 같은 글자들을 제외하고 'z' 보다 그 코드 값이 큰 글자는 알파벳 소문자인데, 이들은 str[i] -= 'a' - 'A';문에 의해 모두 대문자로 바뀐다. 따라서 문제 4-3의 답은 DON6T0이다.

이후 str 배열의 요소 값 중, 현재 값이 6인 len을 기준으로 len 값에 해당하는 위치, 그 2배 및 4배 위치에 각기 '?', '&', 0x40이 저장되는데, 이로 인해 7번째 요소의 값인 널 문자가 '?'로 바뀌고, 13번째 요소의 값인 'e'(believe의 마지막 e에 해당)가 '&'로 바뀌며, 25번째 요소의 값인 널 문자가 0x40(십진수로 64)에 해당하는 '@'로 바뀐다. 이로 인해 str 배열에 저장되어 있던 문자열의 길이가 최초의 길이로 복원된다. 따라서 문제 4-4의 답은

DON6T0?eliev&34anything3@you read in this book.

이다.

이 후, str 배열에 저장된 문자열에서 마지막 문자로부터 거꾸로 읽어 '@'인 첫 문자를 찾고, 방향을 바꿔 순방향으로 이 문자 다음 문자부터 글자들을 복사하여 str2에 저장한다. 마지막 문자인 '.'을 복사한 후, 그 다음 요소에 널 문자를 저장하여 문자열을 완성하지 않더라도, str2에 저장된 내용이 하나의 문자열이 되는 이유는 str2를 선언할 때, char str2[80] = { 0 };와 같이 초기화함으로써, 80개의 요소에 모두 0(널 문자)이 이미 저장되어 있기 때문이다. 따라서 문제 4-5의 답은

you read in this book.

이다.

상기 문자열 복사가 종료되었을 때, pd는 str2에 저장된 문자열의 마지막 부분에 있는 널 문자를 가리키고 있는 상태에 있다. 따라서 *(pd-- - 5)는 str2에 저장된 문자열의 내용 중에서 마지막에서 5번째 글자인 b에 해당한다(pd--는 후감소 연산임에 주의). 물론 str2[4]는 str2에 저장된 문자열의 내용 중에서 처음에서 5번째 글자인 r에 해당한다. 따라서 문제 4-6의 답은 b r이다.

문제 5. (20점) 아래 소스 프로그램과 그 실행 예를 보고 아래 물음에 답하라.

```
#include <stdio.h>

void func(int num)
{
    int tmp;
    int i, flag = 0;

    for (i = [redacted]; i >= 0; i--)
    {
        tmp = [redacted];
        if ([redacted])
        {
            printf("%d", 1);
            flag = 1;
        }
        else
            if ([redacted])
                printf("%d", 0);
    }
    printf("\n");
}

void main(void)
{
    int num;

    while (1)
    {
        printf("10진수 정수 입력: ");
        scanf("%d", &num);
        if (num == 0)
            break;
        func(num);
    }
}
```

문제 5-1

문제 5-2

문제 5-3

문제 5-4

Microsoft Visual Studio 디버그 콘솔

```
10진수 정수 입력:
11111111111111111111111111111111
10진수 정수 입력: 1
1
10진수 정수 입력: 255
11111111
10진수 정수 입력: 100
1100100
10진수 정수 입력: -255
11111111111111111111111100000001
10진수 정수 입력: 0

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로그래밍
 인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 5-5

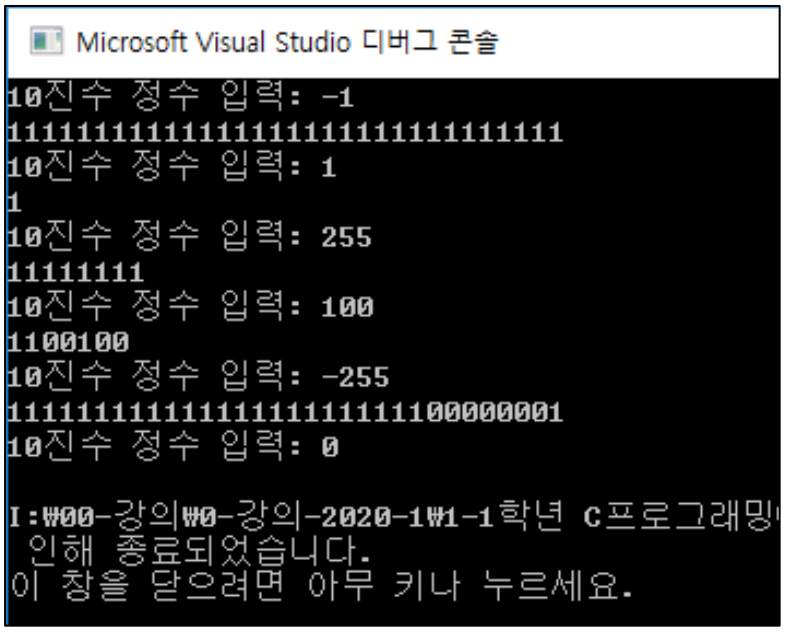
<u>물음</u>:

문제 5-1 ~ 문제 5-4: 가려진 부분에 들어 갈 <u>C 언어 표현(expression)</u>을 각 문제 별로 답하라.

문제 5-5: 가려진 부분에서 사용자가 <u>2글자</u>를 입력하고 엔터 키를 쳤다. 이 2글자를 답하라.

문제 5 끝.

답:



문제 5-1의 답: `sizeof(int) * 8 - 1`

문제 5-2의 답: `1 << i`

문제 5-3의 답: `num & tmp` 또는 `tmp & num`

문제 5-4의 답: `flag` 또는 `flag == 1`

문제 5-5의 답: `-1`

채점 기준:

각 소문항 당 4점 배점. 문제 5-3과 문제 5-4의 경우, 두 가지 중 한 가지라도 답하면 맞는 답임. 문제 5-1 ~ 문제 5-3의 경우, 답에 들어 있는 빈칸은 없더라도 맞는 답임. 문제 5-1 ~ 문제 5-4의 경우, 상기 답 이외의 답이라 하더라도, 알고리듬 상으로 동일한 작업을 수행하게 하는 답이면 맞는 답임. 문제 5-1의 경우, 답으로서 31이라는 값을 직접 적거나 4 * 8 – 1 등의 형태로 적은 것도 맞는 답으로 처리함. 다른 부분 점수 없음

풀이:

문제의 지문 중 출력 내용으로부터 추정하면 문제의 지문의 프로그램이 수행하는 일은 프로그램 사용자로부터 부호가 있는 십진수를 입력 받아, int형 변수에 저장한 후, 이 변수의 내용을 MSB(Most Significant Bit)부터 LSB(Least Significant Bit)까지 모두 출력하되 앞서 나오는 0들(leading 0s)은 출력하지 않은 작업을 반복적으로 수행하되 프로그램 사용자가 0을 입력하면 프로그램을 종료하는 것이다.

만약 프로그램 사용자가 음수를 입력하면 int형 변수 num에 2의 보수 형태로 저장되기 때문에 VS2017의 경우 int형은 4바이트 메모리 공간을 차지하므로 sizeof(int) x 8 = 32비트를 출력하게 된다.

프로그램에서 func 함수 내 for문에서 num 변수의 MSB부터 시작하여 LSB까지 1회 반복할 때 마다 num 변수의 1비트를 처리하므로 이 for문의 반복 횟수는 32회가 되어야 하는데, 이 for문의 조건식이 i가 0일 때에도 참이 되도록 나와 있으므로, 초기식에서의 초기값은 31이 되어야 한다.

따라서 문제 5-1의 답은 **`sizeof(int) * 8 - 1`**이다.

func 함수 내의 for문의 반복영역에서 num 변수가 활용되는 부분이 나와 있지 않으므로 tmp에 대입될 값 및/또는 tmp가 등장하는 문장 바로 아래의 if문의 조건식에 num 변수가 개입되어 있어야 함을 알 수 있다. tmp가 등장하는 문장 바로 아래의 if문의 if 블록에서는 1을 1개 출력하고 else 블록에서는 0을 1개 출력하는 역할을 하므로, 이 if문의 조건식은 이번에 출력할 num의 1비트가 1이면 참이 되어야 함을 알 수 있다. 이와 같은 점을 모두 고려하면, tmp에 num의 검사 대상 비트 위치에만 1인 값을 저장한 후 if문의 조건식을 num과 tmp를 비트단위 AND하면 소기의 목적을 거둘 수 있음을 알 수 있다. (물론 이 방법 이외에 알고리듬적으로 동일한 작업을 수행하게 하는 답이 있을 수도 있다.) tmp에 num의 검사 대상 비트 위치에만 1인 값을 저장하는 한 가지 방법은 1을 i번 좌로 시프트하는 것이다. 따라서 문제 5-2의 답은 **`1 << i`**이고, 문제 5-3의 답은 **`num & tmp`**이다.

상기 언급한 else 블록에서는 leading 0s에 속하지 않을 때 0을 출력해야 하므로, 이를 위해 flag 변수가 사용되었음을 알 수 있다. 즉 flag는 0으로 초기화되었다가 처음으로 1값에 해당하는 비트를 출력하면 flag가 1로 변경되므로, 이 else 블록 내의 if문의 조건식은 flag가 1이면 참이 되어야 한다. 조건식의 값이 0이 아니면 참으로 간주되므로 flag 자체만 조건식에 넣어도 된다. 따라서 문제 5-4의 답은 **`flag`** 또는 **`flag == 1`**이 된다.

문제의 지문의 출력 내용 중, 두 번째 줄을 보면 32비트 모두 1로 출력되어 있으므로, 프로그램 사용자가 입력한 값은 음수이다. 따라서 출력된 32비트를 모두 반전시킨 후 1을 더하면 프로그램 사용자가 입력한 값의 절대값을 얻는다. 그 결과는 1이므로 프로그램 사용자가 입력한 내용은 -1 두 글자이다. 따라서 문제 5-5의 답은 **`-1`**이다.

전체 끝.